Tier 2 / Real servers

Bound data, work and interactive retries.

Storage

Use fixed SQL and bound parameters. Limit rows and duration.

Keep filesystem reads inside a resolved allowed root; reject symlinks/escape, oversized files and unsupported types.

Use explicit idempotency receipts for safely replayable writes.

Progress and cancellation

Progress uses a request-chosen token. It does not replace a final result.

Propagate cancellation and release work in finally blocks. A timeout does not prove an operation had no effect.

Interactive input

An input_required result embeds elicitation/create requests.

The client asks the person, then retries with a fresh ID, exact requestState and keyed inputResponses.

Bind state to arguments and principal; consume once; expire it. Decline must not mutate data.

HTTP

Use Streamable HTTP. Let the SDK derive matching version, method and name headers.

Bind to 127.0.0.1 by default. Validate Host and exact Origin; configure CORS separately.

Use request-body limits and deadlines. A public service also needs HTTPS and authorization.

Run and inspect

uv run python scripts/tier2.py

uv run python scripts/http_lab.py --help

Capture real messages with scripts/wiretap.py; captures redact credentials.

Checkpoint

Demonstrate read bounds, denied writes, idempotent replay, cancellation cleanup and Origin rejection.

Next: Personal Knowledge capstone and Tier 3.